

Task Master

Project - Iteration I

SOEN 342: Software Requirements & Deployment

Professor Ali Jannatpour

Due: 27/02/2026

M-Amar Kseibi
40276594

Anthony Vaccaro
40214876

Ariberto Bello Veras
40319600

System Operations & Operation Contracts

I. System Operations

A. Scope

This section defines the **System Operations** for the *Personal Task Management System* at a black-box level (operations the external actor can invoke on the system). The operations listed here represent the system's interface for supporting the required functionality, including:

- Creating and updating **Tasks**, and managing task attributes (title, description, priority, due date, status).
- Creating and managing **Projects**, and associating tasks to at most one project.
- Creating and managing **Subtasks** that belong to exactly one parent task.
- Creating and applying **Tags** to tasks, and removing tag associations.
- Retrieving information through **query operations**, such as listing/filtering tasks and searching tasks.
- Viewing **Activity History** information recorded for task lifecycle events (e.g., created, updated, assigned, completed, cancelled).

System operations are grouped into:

- **Commands** (state-changing operations): create/update/associate/detach objects.
- **Queries** (read-only operations): list/search/view operations that do not modify system state.

Unless stated otherwise, successful execution of state-changing operations results in the system state being updated consistently with the domain model rules (e.g., a task belongs to at most one project; subtasks cannot exist without a parent task).

B. Critical Operations

ID	Operation	Type	Parameters	Brief Description
SO1	createTask(title, description, priority)	Command	title, description, priority	Creates a new task with the given attributes and logs activity.
SO2	updateTask(taskID)	Command	taskID	Updates an existing task identified by taskID and logs activity.
SO3	createProject(name, description)	Command	name, description	Creates a new project with the given name/description.
SO4	assignTaskToProj(taskID, projectID)	Command	taskID, projectID	Assigns a task to a project (task belongs to at most one project).

C. Command Operations

ID	Operation	Type	Parameters	Brief Description
SO5	detachTaskFromProj(taskTitle)	Command	taskTitle	Removes a task from its current project association.
SO6	switchTaskToProj(taskTitle, projName)	Command	taskTitle, projName	Moves a task from its current project to another project.
SO7	createSubtask(parentTaskTitle, subTaskTitle)	Command	parentTaskTitle, subTaskTitle	Creates a subtask under a parent task.
SO8	createTag(name)	Command	name	Creates a new tag.
SO9	addTagToTask(taskTitle, tag)	Command	taskTitle, tag	Associates an existing tag with a task.
SO10	removeTagFromTask(taskTitle, tag)	Command	taskTitle, tag	Removes a tag association from a task.

D. Query Operations

ID	Operation	Type	Parameters	Brief Description
SO11	listTasks()	Query	—	Returns all tasks in the system.
SO12	listTasksByDueDate(date)	Query	date	Returns tasks due on the specified date.
SO13	listTasksByRangeDate(startingDate, endingDate)	Query	startingDate, endingDate	Returns tasks due within the given date range.
SO14	listTasksByPriority()	Query	— (or priority filter implied)	Returns tasks ordered/filtered by priority.
SO15	listTasksByStatus(status)	Query	status	Returns tasks filtered by status.
SO16	listTasksByProject(projectName)	Query	projectName	Returns tasks associated with a given project.
SO17	listTasksByTag(tag)	Query	tag	Returns tasks associated with a given tag.
SO18	searchTaskByTitle(taskTitle)	Query	taskTitle	Returns tasks matching a title keyword or exact title.
SO19	searchTaskByDesc(description)	Query	description	Returns tasks matching a description keyword.
SO20	viewActivityHistory()	Query	— (task context implied)	Returns the activity history (typically for a selected task).

II. Operations Contracts

A. Contract Format

Each contract describes:

- **Operation**
- **Preconditions** (must be true before execution)
- **Postconditions** (state changes after successful execution)
- **Failure Conditions** (common reasons the operation is rejected)

CO1 – **createTask**

Operation: `createTask(title, description, priority)`

Preconditions

- The user has requested to create a task.
`title` is provided and non-empty.
`priority` is provided and valid (per system's priority definition).

Postconditions (Success)

- A new **Task** object `t` is created.
- A unique `taskID` is generated and assigned to `t`.
- `t.title = title`
- `t.description = description` (*may be empty/null if allowed*)
- `t.priority = priority`
- `t.creationDate` is set to the current system date/time.
- `t.status` is set to **open**.
- A new **ActivityEntry** `a` is created for `t`:
 - `a.timestamp = now`
 - `a.description` indicates the task was created (e.g., "Task created")
- `a` is associated with `t` (added to the task's activity history collection).

Failure Conditions

- If `title` is empty → operation rejected, no task created.
- If `priority` is missing/invalid → operation rejected, no task created.

CO2 – updateTask

Operation: `updateTask(taskID)`

Preconditions

- A task with id `taskID` exists.
- The user has provided **at least one valid change** to apply (e.g., title/description/priority/due date/status/project/tags as allowed by the use case).

Postconditions (Success)

- The specified task `t` (identified by `taskID`) is updated according to the requested changes.
- Only the attributes included in the request are modified; all other task attributes remain unchanged.
- A new **ActivityEntry** `a` is created for `t`:
 - `a.timestamp = now`
 - `a.description` summarizes the update (e.g., “Task updated: priority changed”)
- `a` is associated with `t` (added to task history).

Failure Conditions

- If no task exists with `taskID` → operation rejected, no changes occur.
- If no valid changes are provided → operation rejected, no changes occur.

CO3 – createProject

Operation: `createProject(name, description)`

Preconditions

- The user has requested to create a project.
- `name` is provided and non-empty.

Postconditions (Success)

- A new **Project** object `p` is created.
- A unique `projectID` is generated and assigned to `p`.
- `p.name = name`
- `p.description = description` (*may be null/empty*)

Failure Conditions

- If `name` is empty → operation rejected, no project created.
- If a uniqueness rule exists for project names and `name` already exists → operation rejected.
(*Optional rule—include only if your team uses it.*)

CO4 – assignTaskToProj

Operation: `assignTaskToProj(taskID, projectID)`

Preconditions

- A task with id `taskID` exists.
- A project with id `projectID` exists.
- The task is not currently assigned to any other project (*i.e., no project association exists*).

Postconditions (Success)

- The task `t`'s project association is set to the project `p` identified by `projectID`.
- The task is now considered **assigned to a project** and remains associated with **at most one** project.
- The task is added to the project's collection/list of tasks.
- An **ActivityEntry** is created for the task with `timestamp = now` and description indicating project assignment.

Failure Conditions

- If no task exists with `taskID` → operation rejected, no changes occur.
- If no project exists with `projectID` → operation rejected, no changes occur.
- If the task is already assigned to a project → operation rejected under this contract (your `switchTaskToProj` handles reassignment).